

Assignment-1

1. Introduction to Python and its Features (simple, high-level, interpreted language).

What is Python?

Python is a widely used **high-level, interpreted programming language** created by Guido van Rossum. It is designed to be simple, readable, and easy to learn, making it popular among beginners as well as professionals.

Features of Python:

- **Simple and Easy to Learn**
Python has a clean and readable syntax that looks similar to English. This makes it easier to write and understand code compared to many other languages.
- **High-Level Language**
Python is a high-level language, which means you don't need to manage complex details like memory allocation. It focuses more on problem-solving rather than system-level operations.
- **Interpreted Language**
Python code is executed line by line using an interpreter, without needing compilation. This makes debugging easier and speeds up development.
- **Platform Independent**
Python programs can run on different operating systems like Windows, macOS, and Linux without modification.
- **Dynamically Typed**
You don't need to declare the type of a variable explicitly. Python automatically determines the data type at runtime.
- **Extensive Libraries and Frameworks**
Python has a huge collection of built-in libraries and frameworks for various tasks such as web development, data science, AI, and automation.
- **Object-Oriented and Flexible**
Python supports multiple programming paradigms, including object-oriented, procedural, and functional programming.

2. History and evolution of Python.

- Python was created by Guido van Rossum and released in **1991**.
It was designed to be a **simple, readable, and easy programming language**.
- Over time, Python improved by adding new features, better performance, and wider usage.
Today, it is used in many fields like web development, AI, and automation.
- It is maintained by Python Software Foundation.
- Over time, Python became very popular because:
It is easy to read, simple to write, useful in many fields.

3. Advantages of using Python over other programming languages.

- **Easy to Learn and Use**
Python has simple and readable syntax, so beginners can learn it quickly.
- **Less Code, More Work**
You can do tasks with fewer lines of code compared to other languages like Java or C++.
- **Interpreted Language**
Python runs line by line, so it is easier to test and debug programs.
- **Platform Independent**
Python code can run on Windows, macOS, and Linux without changes.
- **Large Library Support**
Python has many built-in libraries for web development, data science, AI, etc.
- **Supports Multiple Programming Styles**
You can write code using object-oriented, procedural, or functional styles.
- **Strong Community Support**
Python has a large community that helps with learning and problem-solving.
- **Widely Used**
Python is used in many areas like:
 - Web development
 - Artificial Intelligence
 - Automation

4. Installing Python and setting up the development environment (Anaconda, PyCharm, or VS Code).

1. Install Python

- Go to the official website: Python
- Download the latest version
- Run the installer
- Add python to path"
- Click Install

2. Choose a Development Environment

VS Code (Lightweight & Popular)

Visual Studio Code

- Install VS Code
- Install Python extension
- Open folder ->Create.py file ->Run code

5. Writing and executing your first Python program

```
print("Hello, Python!")
```

- Save file with .py extension
- Run the code
- Output : Hello, Python!

6. Understanding Python's PEP 8 guidelines

Python's official style guide, PEP 8 (Python Enhancement Proposal 8), defines conventions for writing clean and consistent Python code. It covers several aspects such as naming conventions, indentation, line length, and more.

- Indentation: Use 4 spaces per indentation level (avoid using tabs).
- Maximum Line Length: Limit all lines to a maximum of 79 characters (72 for docstrings).
- Naming Conventions
Python has specific conventions for naming variables, functions, classes, and modules. These conventions help make code more readable and consistent.
- Classes: Use CamelCase (first letter capitalized, no underscores)

7. Indentation, comments, and naming conventions in Python.

➤ Indentation

- Indentation means spaces at the beginning of a line
- In Python, indentation is very important (it defines blocks of code)

➤ Comments

- Comments are used to explain code
- Python ignores comments during execution

Types of comments:

- Single-line comment
- Multi-line comment

➤ Naming Conventions

- Rules for naming variables, functions, etc.

Basic rules:

- Use letters, numbers, and underscore (_)
- Must start with a letter or underscore
- No spaces allowed
- Case-sensitive (name ≠ Name)

8. Writing readable and maintainable code.

```
# Take name from user
name = input("Enter your name: ")
print("Name is", name) # Display name
```

```
# Take age from user
age = input("Enter your age: ")
print("Age is", age) # Display age
```

```
# Take date of birth from user
dob = input("Enter your DOB (DD/MM/YYYY): ")
print("DOB is", dob) # Display DOB
```

```
# Take mobile number from user
mobile = input("Enter your mobile: ")
print("Mobile no is", mobile) # Display mobile number
```

Explanation of Comments

- Lines starting with # are comments
 - Comments are used to explain the code
 - Python ignores comments when running the program
-
- # used for comments
 - Helps make code easy to understand

9. Understanding data types: integers, floats, strings, lists, tuples, dictionaries, sets

- `int` (Integer): Represents whole numbers, positive or negative, without decimals.
- `float` (Floating-point): Represents real numbers or numbers with a fractional part.
- `str` (String): Represents a sequence of characters enclosed in single, double, or triple quotes
- `list`: Represents an ordered, mutable collection of items. Lists can hold different types of data (integers, strings, other lists, etc.).
- `tuple`: Represents an ordered, immutable collection of items. Once created, its elements cannot be changed.
- `dict` (Dictionary): Represents a collection of key-value pairs. Each key is unique, and the associated value can be any data type.
- `set`: Represents an unordered collection of unique elements. Sets do not allow duplicate values.

10. Python variables and memory allocation

A variable in Python is a name used to store data or values. Instead of holding the actual value, a variable refers to a location in memory where the data is stored.

When a variable is created:

- Python stores the value in memory
- It assigns a memory location (address) to that value
- The variable name is linked to that memory location

Python uses dynamic typing, which means you do not need to declare the data type of a variable. The type is automatically decided at runtime.

Variables in Python are references (labels) to objects, not the actual data itself. Multiple variables can refer to the same value in memory.

Python also has automatic memory management:

- It allocates memory when needed
- It removes unused data using garbage collection

11. Python operators: arithmetic, comparison, logical, bitwise.

➤ Arithmetic Operators

Arithmetic operators are used to perform mathematical calculations such as addition, subtraction, multiplication, division, etc.

➤ Comparison Operators

Comparison operators are used to compare two values.

They return a result in the form of True or False.

➤ Logical Operators

Logical operators are used to combine multiple conditions.

They help in making decisions based on more than one condition.

➤ Bitwise Operators

Bitwise operators are used to perform operations on binary (bits) values of numbers.

They work at the bit level (0 and 1).

12. Introduction to conditional statements: if, else, elif.

➤ if Statement

The if statement checks a condition.

If the condition is True, the code inside it runs.

➤ else Statement

The else statement runs when the condition in if is False.

➤ elif Statement

The elif (else if) statement is used to check multiple conditions.

If the first condition is false, it checks the next one.

13. Nested if-else conditions.

A nested if-else means using an if or else statement inside another if or else.

It is used when we need to check multiple conditions step by step.

- The program first checks the outer condition
- If it is True, it goes inside and checks the inner condition
- If the outer condition is False, it skips the inner block and executes the outer else
- Each level of nesting adds another layer of decision-making

14. Introduction to for and while loops.

➤ for Loop

A for loop is used when you know how many times you want to repeat something.

- It works on a sequence (like list, string, range)
- It runs a fixed number of times

Example : repeat 5 times

➤ while Loop

A while loop is used when you repeat code until a condition becomes False.

- It depends on a condition
- It may run unknown number of times

Example : run until a number becomes 0

15. How loops work in Python.

Loops in Python are used to repeat a block of code again and again until a condition is met.

Working of Loops

- Start : The loop begins with a condition or a sequence
- Check Condition : Python checks whether the condition is True or False
- Execute Code : If the condition is True, the code inside the loop runs
- Repeat : After execution, Python again checks the condition

This process repeats

- Stop : When the condition becomes False, the loop stops

Types of Loop Working

- for Loop
- while Loop

16. Using loops with collections (lists, tuples, etc.).

loops are often used to go through collections like lists, tuples, strings, sets, and dictionaries.

➤ What is a Collection?

A collection is a group of items, such as:

- List
- Tuple

How Loops Work with Collections:

A loop takes each item from the collection one by one and executes the same code for each item.

Using for Loop:

- The for loop is mainly used with collections
- It automatically picks each element one by one

Works like:

- Take first item : run code
- Take next item : run code
- Continue until all items are finished

Collections and Loop Behavior:

- List / Tuple / Set -> loop goes through each element
- String -> loop goes through each character
- Dictionary -> loop goes through keys (by default)

17. Understanding how generators work in Python.

A generator in Python is a special type of function that returns values one at a time, instead of returning all values at once.

Generators use the keyword `yield` instead of `return`. When a generator function is called, it does not execute immediately. Instead, it returns a generator object.

How Generators Work:

- When the generator is called, it creates an iterator object

- The function runs only when needed (on demand)
- Each time yield is reached, it:
 - Sends a value back
 - Pauses the function
- When called again, it resumes from where it stopped
- This continues until all values are generated

18. Difference between yield and return.

➤ return

- Ends the function completely
- Returns a single value
- Function stops execution after return

➤ yield

- Does not end the function
- Returns values one by one
- Function pauses and resumes from same point

19. Understanding iterators and creating custom iterators.

➤ What is an Iterator?

An iterator is an object that allows you to traverse (go through) elements one by one.

In Python:

- Lists, tuples, strings -> all are iterable
- An iterator gives elements one at a time

➤ What is a Custom Iterator?

A custom iterator is when you create your own iterator using a class.

You define:

- How iteration starts
- How next value is generated

20. Defining and calling functions in Python

➤ What is a Function?

A function in Python is a reusable block of code that performs a specific task. Instead of writing the same code multiple times, you define it once and call it whenever needed.

➤ Types of Functions (Based on Arguments & Return)

- No parameters, no return value
- Parameters, no return value
- No parameters, with return value
- Parameters, with return value

➤ Calling a Function

Calling a function means executing it.

➤ Syntax:

```
function_name(arguments)
```

- The function runs when it is called.
- If it has parameters, you must pass arguments

21. Function arguments (positional, keyword, default)

Function arguments are values passed to a function when it is called. They allow functions to work with different data inputs.

➤ Positional Arguments

- These are the most basic type of arguments.
- Values are passed in the same order as the function parameters.
- The position of each argument matters.

Key Points:

- Number of arguments must match the number of parameters.
- Order must be correct.

➤ Keyword Arguments

- Arguments are passed using parameter names (keywords).
- Order does not matter because values are assigned based on names.

Key Points:

- Improves readability.
- You explicitly specify which value goes to which parameter.
- Can be used in any order.

➤ Default Arguments

- Parameters are given default values in the function definition.
- If no value is provided during the function call, the default value is used.

Key Points:

- Makes parameters optional.
- Default values are assigned only once at function definition time.
- Non-default parameters must come before default parameters.

22. Scope of variables in Python

Scope refers to the region of a program where a variable is accessible. In Python, not all variables can be used everywhere—access depends on where they are defined.

Types of Variable Scope:

➤ Local Scope

- A variable defined inside a function.
- It can only be accessed within that function.
- It is created when the function is called and destroyed after execution.

Key Idea: Limited to the function block.

➤ Global Scope

- A variable defined outside all functions.
- It can be accessed throughout the program (inside and outside functions).

Key Idea: Available everywhere in the file.

➤ Enclosing Scope (Non-local Scope)

- Occurs in nested functions (a function inside another function).
- The inner function can access variables from the outer (enclosing) function.

Key Idea: Between local and global scope.

➤ Built-in Scope

- The outermost scope in Python.
- Contains predefined names provided by Python (like functions and keywords).
- Accessible from anywhere in the program.

Key Idea: Always available.

23. Built-in methods for strings, lists, etc.

Python provides built-in methods for different data types like strings, lists, tuples, sets, and dictionaries. These methods are predefined functions that help perform common operations easily.

What are Built-in Methods?

- Functions that are already defined in Python.
- They are called using the dot (.) operator on an object.
- Each data type has its own set of methods.

➤ String Methods

String methods are used to manipulate and analyze text data.

Key Concepts:

- Strings are immutable (cannot be changed).
- Methods return a new string instead of modifying the original.

Common Categories:

- Case conversion (upper, lower, title)
- Searching (find, index)
- Checking (isalpha, isdigit, islower)
- Formatting and splitting (split, join, strip, replace)

➤ List Methods

List methods are used to manage collections of items.

Key Concepts:

- Lists are mutable (can be changed).
- Methods modify the original list in most cases.

Common Categories:

- Adding elements (append, extend, insert)
- Removing elements (remove, pop, clear)
- Sorting and reversing (sort, reverse)
- Searching (index, count)

➤ Tuple Methods

Tuples are similar to lists but immutable.

Key Concepts:

- Limited number of methods due to immutability.

Common Methods:

- count()
- index()

➤ Set Methods

Sets store unique elements and are unordered.

Key Concepts:

- No duplicate values allowed.
- Useful for mathematical operations.

Common Categories:

- Adding/removing (add, remove, discard)
- Set operations (union, intersection, difference)
- Checking relationships (issubset, issuperset)

➤ Dictionary Methods

Dictionaries store data in key-value pairs.

Key Concepts:

- Keys must be unique.
- Values can be of any type.

Common Categories:

- Accessing data (get, keys, values, items)
- Modifying data (update, pop, popitem)
- Utility methods (clear, copy)

24. Understanding the role of break, continue, and pass in Python loops.

loops (for and while), control statements like break, continue, and pass are used to alter the normal flow of execution.

➤ break Statement

- The break statement is used to terminate the loop immediately.
- When break is encountered, the loop stops executing and control moves to the next statement after the loop.

Key Points:

- Works in both for and while loops
- Often used when a specific condition is met
- Helps avoid unnecessary iterations

➤ continue Statement

- The continue statement is used to skip the current iteration of the loop.
- The loop does not stop; it simply moves to the next iteration.

Key Points:

- Useful when you want to ignore certain conditions

- Only skips one iteration, not the entire loop
- The loop continues running normally afterward

➤ pass Statement

- The pass statement is a null operation (does nothing).
- It is used as a placeholder where a statement is syntactically required but no action is needed.

Key Points:

- Prevents syntax errors in empty blocks
- Used during code development (when logic is not yet implemented)
- Does not affect loop execution

25. Understanding how to access and manipulate strings.

Accessing Strings:

➤ Indexing

- Each character in a string has a position (index).
- Indexing starts from 0 for the first character.
- Negative indexing is also supported (starts from the end with -1).

Key Idea: Access a single character using its index.

➤ Slicing

- Used to access a range of characters from a string.
- Defined using start, stop, and step values.

Key Idea: Extract a portion of a string without modifying the original.

➤ Iteration

- Strings can be accessed character by character using loops.
- Useful for processing each character individually.

Manipulating Strings:

➤ Immutability

- Strings are immutable, meaning they cannot be changed directly.
- Any modification creates a new string.

➤ Concatenation

- Combining two or more strings into one.
- Done using operators or methods.

➤ Repetition

- A string can be repeated multiple times.
- Useful for patterns or formatting.

➤ String Methods

- Python provides many built-in methods to manipulate strings.
- These include:
 - Changing case (uppercase/lowercase)
 - Removing spaces
 - Replacing text
 - Splitting and joining strings

➤ Membership Checking

- Used to check if a substring exists within a string.

➤ Formatting Strings

- Allows inserting values into strings dynamically.
- Improves readability and presentation of output.

26. Basic operations: concatenation, repetition, string methods (upper(), lower(), etc.).

Strings in Python support several basic operations that allow you to combine, modify, and analyze text efficiently.

➤ Concatenation

- Concatenation means joining two or more strings into one.
- It is used to combine text from different sources.

Key Points:

- Creates a new string (original strings remain unchanged)
- Useful for building messages or dynamic text

➤ Repetition

- Repetition means repeating a string multiple times.
- Often used for patterns or formatting.

Key Points:

- Produces a new string with repeated content
- Helpful for generating repeated sequences

String Methods:

String methods are built-in functions used to manipulate and analyze strings.

➤ Case Conversion Methods

- upper() -> Converts all characters to uppercase
- lower() -> Converts all characters to lowercase
- title() -> Capitalizes the first letter of each word
- capitalize() -> Capitalizes the first character of the string

➤ Whitespace Handling

- strip() -> Removes spaces from both ends
- lstrip() -> Removes spaces from the left

- `rstrip()` -> Removes spaces from the right

➤ Searching Methods

- `find()` -> Returns the index of a substring
- `index()` -> Similar to `find()` but gives an error if not found

➤ Replacement and Modification

- `replace()` -> Replaces part of a string with another
- `split()` -> Splits a string into a list
- `join()` -> Joins elements into a single string

➤ Checking Methods

- `isalpha()` -> Checks if all characters are alphabets
- `isdigit()` -> Checks if all characters are digits
- `islower()` / `isupper()` -> Check character case

27. String slicing

String slicing is a technique used to extract a portion (substring) of a string. It allows you to access a range of characters using their index positions.

➤ Basic Concept

- Strings are indexed sequences (starting from 0).
- Slicing lets you retrieve part of the string without changing the original string.

Types of Slicing

➤ Basic Slicing

- Extracts characters between two positions.

➤ From Start or End

- If start is omitted -> begins from index 0
- If stop is omitted -> goes till the end

- Negative Indexing
 - Allows slicing from the end of the string
 - -1 represents the last character
- Step Slicing
 - Skips characters based on step value
 - Useful for patterns (e.g., alternate characters)
- Reverse Slicing
 - Using a negative step reverses the string

28. How functional programming works in Python

Functional programming is a programming paradigm where computation is treated as the evaluation of mathematical functions. It focuses on writing code using pure functions, avoiding changes in state, and minimizing side effects.

In functional programming, instead of changing data step-by-step, you:

- Define functions
- Pass data through these functions
- Get a new result without modifying the original data

Key Concepts of Functional Programming

- Pure Functions
 - A function is pure if:
 - It always produces the same output for the same input
 - It has no side effects (does not modify external variables)
- Immutability
 - Data is not changed after it is created.

- Any “modification” results in a new object instead of altering the original.

➤ First-Class Functions

- Functions in Python are treated as objects.
- They can be:
 - Assigned to variables
 - Passed as arguments
 - Returned from other functions

➤ Higher-Order Functions

- Functions that:
 - Take other functions as arguments
 - Return functions as results

➤ Anonymous Functions (Lambda Functions)

- Small, unnamed functions defined without using def.
- Used for short, simple operations.

➤ Function Composition

- Combining multiple functions to build more complex operations.
- Output of one function becomes input to another.

29. Using map(), reduce(), and filter() functions for processing data.

These are functional programming tools in Python used to process collections of data (like lists, tuples, etc.) efficiently by applying functions.

➤ map() Function

- Used to apply a function to every element in an iterable.
- Returns a new iterable with transformed values.

Key Idea:

- Performs the same operation on all elements.

Characteristics:

- Takes a function and one (or more) iterables
- Does not modify original data
- Produces a mapped result

➤ `filter()` Function

- Used to select elements from an iterable based on a condition.
- Returns only those elements for which the condition is true.

Key Idea:

- Filters out unwanted data.

Characteristics:

- Takes a function that returns True or False
- Keeps elements that satisfy the condition
- Does not change original data

➤ `reduce()` Function

- Used to reduce a sequence to a single value.
- Applies a function cumulatively to elements.

Key Idea:

- Combines all elements step-by-step into one result.

Characteristics:

- Available in the `functools` module
- Processes elements pairwise
- Produces a single output

30. Introduction to closures and decorators.

Closures and decorators are advanced concepts in Python that build on functions as first-class objects and are widely used for writing clean, reusable, and modular code.

Closures:

What is a Closure?

A closure is a function that remembers the variables from its enclosing scope, even after the outer function has finished execution.

Key Idea:

- A function defined inside another function can retain access to the outer function's variables.
- This retained data is called a closure.

Characteristics:

- Involves nested functions
- Inner function uses variables from the outer function
- Outer function returns the inner function
- Preserves state without using global variables

Decorators:

What is a Decorator?

A decorator is a function that modifies or extends the behavior of another function without changing its actual code.

Key Idea:

- Functions are passed as arguments to another function
- The decorator wraps the original function and adds extra functionality

Characteristics:

- Uses higher-order functions
- Commonly written using `@decorator_name` syntax
- Does not modify the original function directly
- Adds reusable functionality

